



Universidad Nacional Autónoma de México  
Facultad de Ingeniería



## **Práctica 3.**

Silverio Martínez Andrés

Lab. Organización y Arquitectura de  
Computadoras

Grupo: 8

Semestre 2026-2

Profesor: Jorge Ferat Toscano

Fecha de entrega: 22 de marzo de 2026

## Práctica 3

### Objetivo

Simular un programa en lenguaje VHDL utilizando el software Quartus II

### Introducción

El diseño de sistemas digitales sobre dispositivos lógicos programables, como los FPGAs, se fundamenta en el uso de bloques esenciales de construcción. Entre ellos destaca el **multiplexor**, un circuito combinacional encargado de canalizar una señal específica hacia una salida única basándose en una señal de control. Esta práctica detalla el diseño e implementación de un multiplexor 2:1 de un bit utilizando **VHDL**. El flujo de trabajo abarca desde la definición de la arquitectura y el comportamiento mediante procesos condicionales, hasta la verificación técnica en **Quartus** mediante el uso del *RTL Viewer* y simulaciones funcionales en el *Waveform Editor*. El objetivo central es consolidar el dominio de la descripción de hardware y validar experimentalmente la lógica de selección de señales.

### Desarrollo

**1. Crear un proyecto con nombre practica 3, utilizando la misma forma de generarlo conforme a lo realizado en la anterior practica 1**

**Paso 1:** Seleccionamos la opción de “New Project Wizard”



**Imagen 1: New Project Wizard**

**Paso 2:** Seleccionamos la carpeta donde queremos guardar el proyecto

**Directory, Name, Top-Level Entity**

What is the working directory for this project?

C:/Users/imano/Documents/Lab. Arquí/Practica 3



**Imagen 2: Directory for this project**

**Paso 3:** Le asignamos el nombre del proyecto

What is the name of this project?

Practica3



What is the name of the top-level design entity for this project? This name is case sensitive and must exactly match the entity name in the design file.

Practica3



Use Existing Project Settings...

**Imagen 3: Name of this Practica3**

**Paso 4:**

**Project Type**

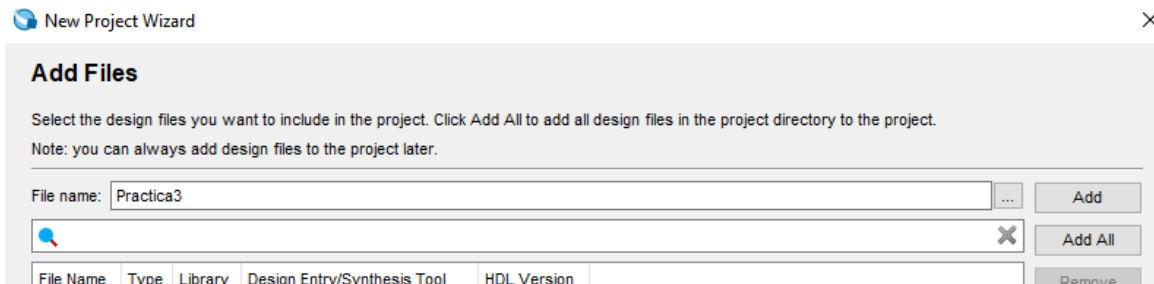
Select the type of project to create.

☒ Empty project

Create new project by specifying project files and libraries, target device family and device, and EDA tool settings.

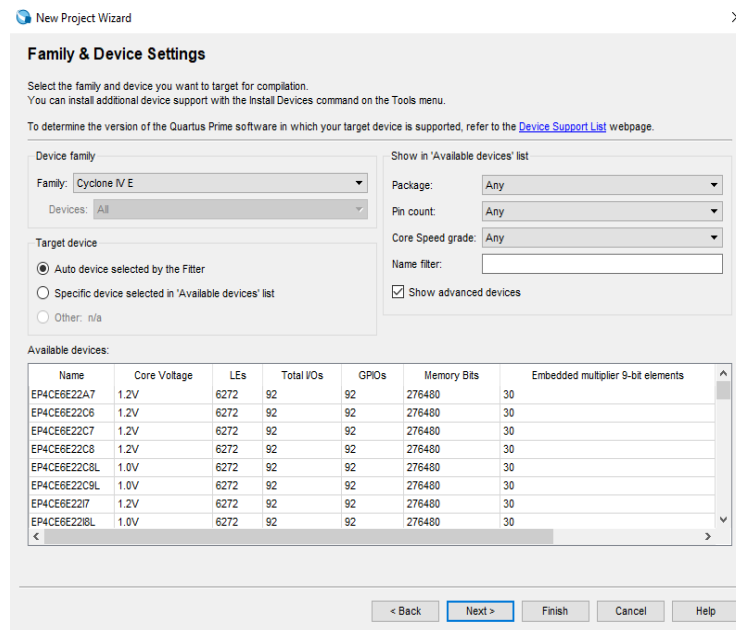
**Imagen 4: Project Type**

## Paso 5: Añadimos el archivo “Practica3”



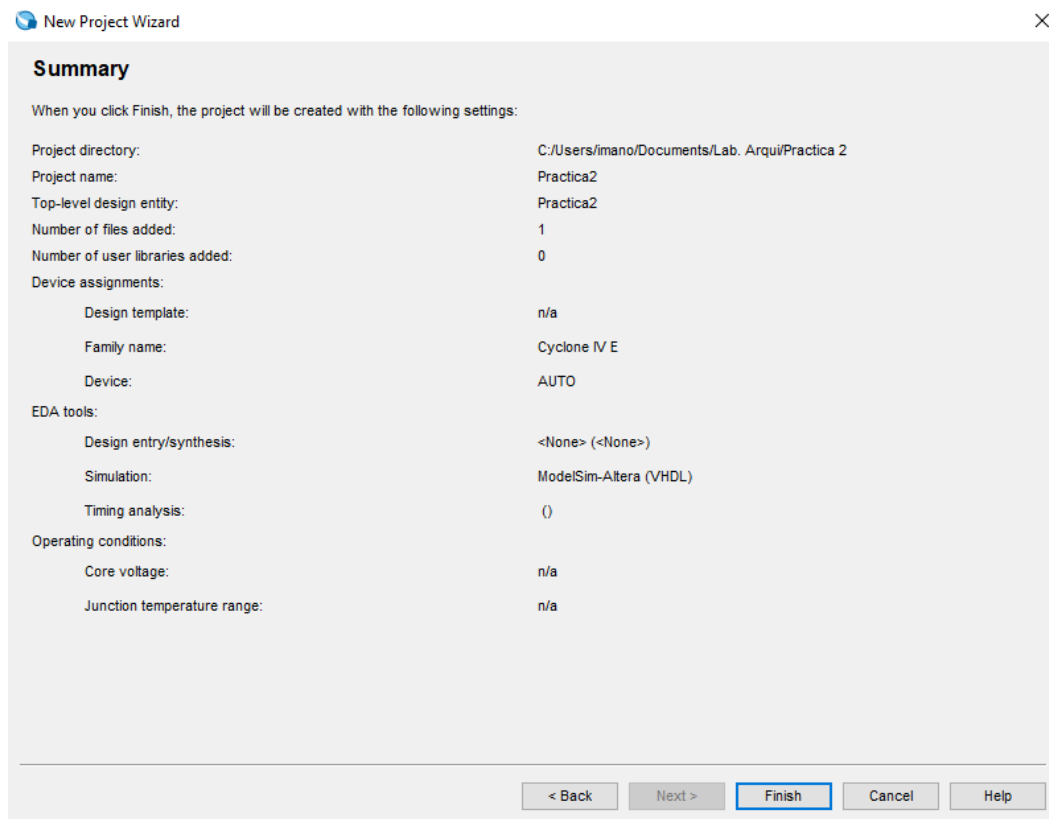
**Imagen 5: Add Files**

## Paso 6: Llegamos a esta ventana y ponemos en auto



**Imagen 6: Family & Device Setting**

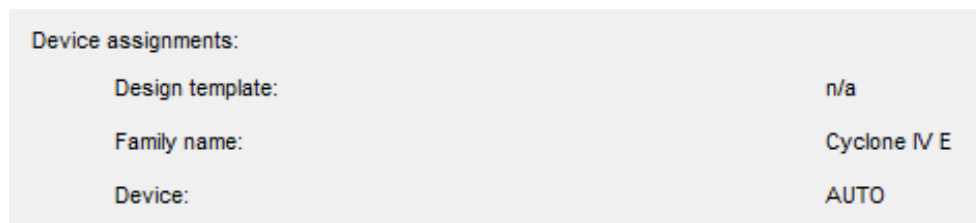
**Paso 7:** Para terminar, nos aparecerá la siguiente ventana y le daremos clic en **“Finish”**



**Imagen 7: Summary**

## **2. El tipo de dispositivo a utilizar considere Auto**

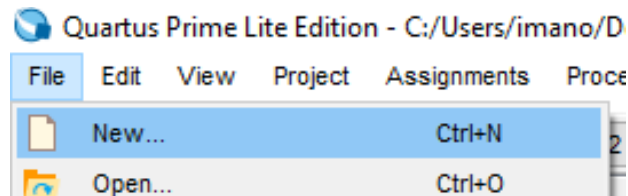
En la siguiente imagen vemos, que pusimos Auto en la configuración:



**Imagen 8: Device AUTO**

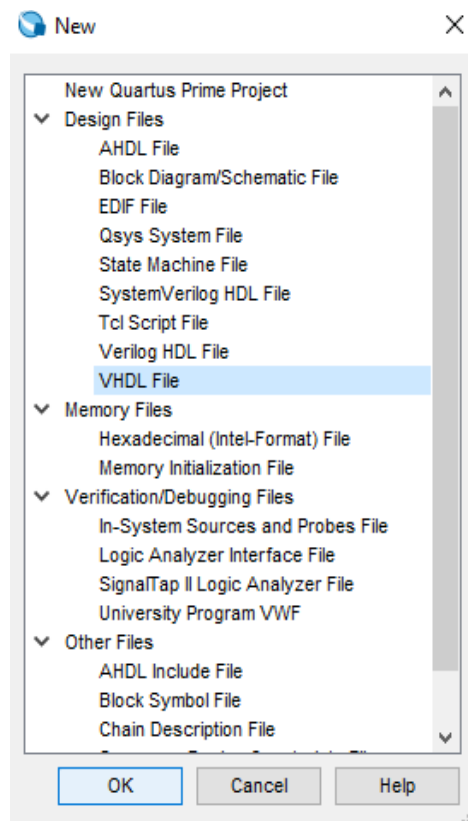
### 3. Crear un archivo nuevo de diseño de tipo VHDL

**Paso 1:** Damos clic en “File” y después en “New”



**Imagen 9: Nuevo archivo**

**Paso 2:** En la siguiente ventana, dar clic en “VHDL File” y después clic en “Ok”



**Imagen 10: VHDL Fi**

#### 4. Una vez que este generado teclear en el editor el código

```
-- Los comentarios empiezan con dos guiones
-- multiplexor de dos líneas de entrada (a y b) de un bit
-- a una sola línea (salida) también de un bit; la
-- señal select sirve para seleccionar la línea
-- seleccionar la línea a (selec = '0') o b (selec = '1')
ENTITY Practica3 IS
PORT ( a: IN bit;
      b: IN bit;
      selec: IN bit;
      salida: OUT bit);
END Practica3;

ARCHITECTURE comportamental OF Practica3 IS
BEGIN PROCESS(a,b,selec)
BEGIN
  IF (selec='0') THEN
    salida<=a;
  ELSE
    salida<=b;
  END IF;
END PROCESS;
END comportamental;
```

**Imagen 11: Código de la practica 3**

#### Explicación:

##### 1. Entidad (ENTITY Practica3)

- Define la interfaz del componente. Tiene:
  - Entradas:** a (bit de entrada 1). b (bit de entrada 2). selec (señal de control).
  - Salida:** salida (bit resultante).

##### 2. Arquitectura (ARCHITECTURE comportamental)

- ARCHITECTURE comportamental:** describe cómo funciona internamente el circuito. **PROCESS(a,b,selec):** indica que el bloque se ejecuta siempre que cambie cualquiera de esas señales.
- Dentro del proceso:
  - Si selec = '0' → la salida toma el valor de **a**.
  - Si selec = '1' → la salida toma el valor de **b**.

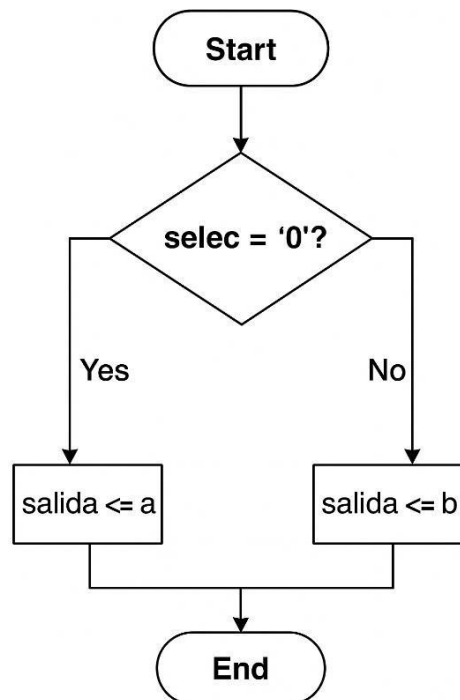
Es la descripción en **alto nivel** del comportamiento de un **multiplexor 2:1**.

### 3. Funcionamiento resumido

- **selec = 0** → **salida = a**
- **selec = 1** → **salida = b**

Es decir, dependiendo del selector, el multiplexor redirige la señal de una de las dos entradas hacia la salida. Este código implementa un **MUX 2 a 1 de 1 bit** en VHDL. Es un bloque fundamental en circuitos digitales, usado para elegir entre dos señales según una señal de control.

**Diagrama de flujo:**

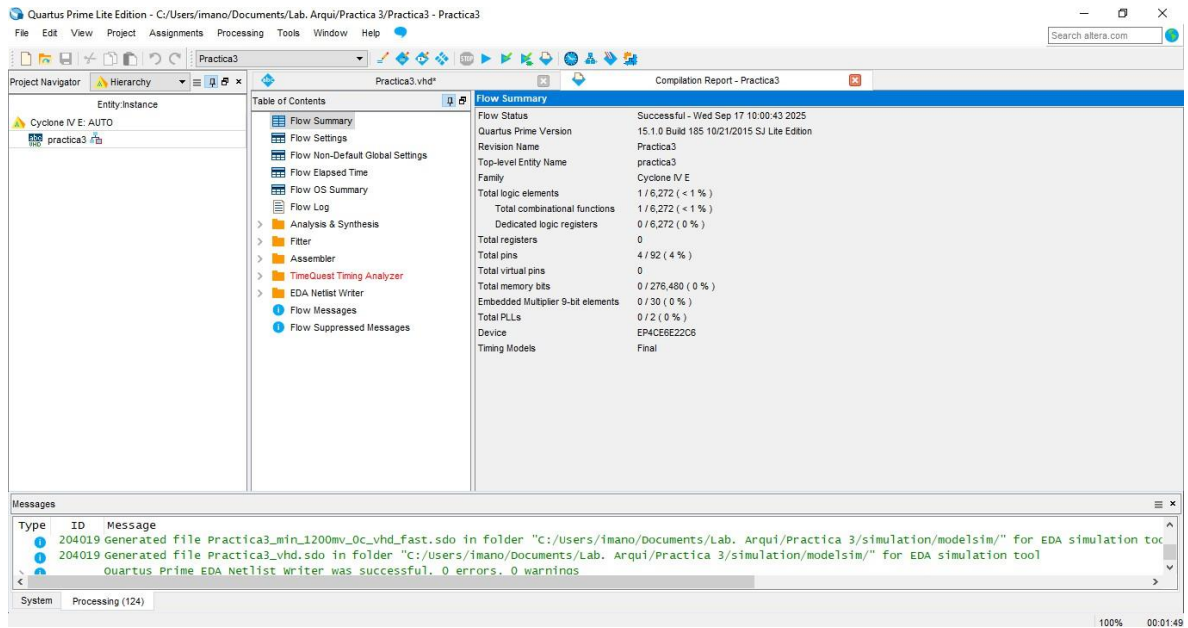


### 5. Compilar el programa

**Revisar que la compilación fue de manera exitosa**

Como vemos en la siguiente imagen, se muestra que fue exitosa la compilación:



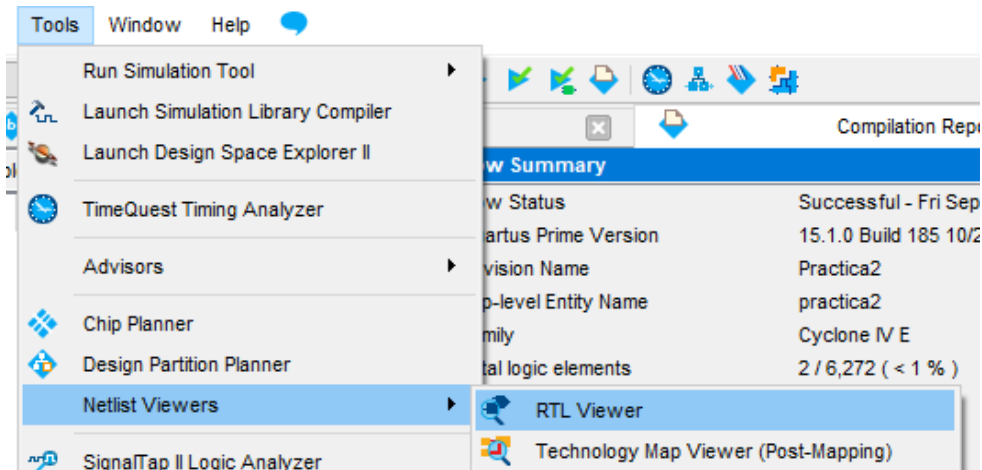


**Imagen 12: compilación del código**

## 6. Procedemos a revisar el circuito sintetizado (RTL Viewer)

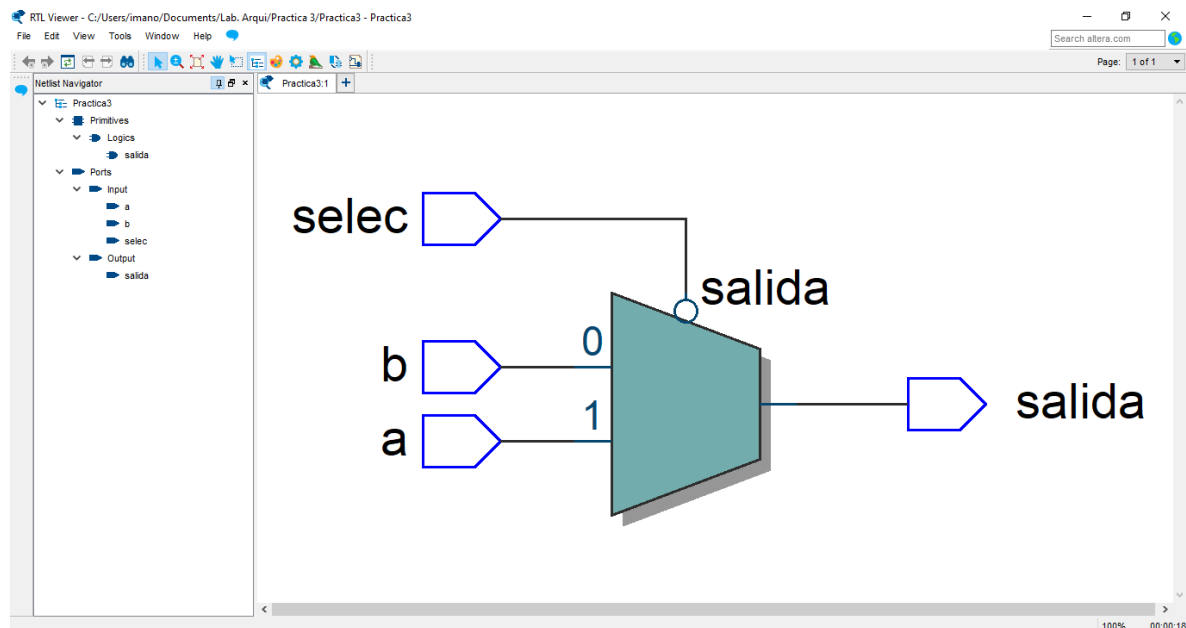
Se observará en la pantalla el diagrama correspondiente al circuito

**Paso 1:** Nos dirigimos en la pestaña de “Tools”, buscamos “Netlist Viewers” y finalmente damos clic en “RTL Viewer”



**Imagen 13: RTL Viewer**

Este es el circuito generado:

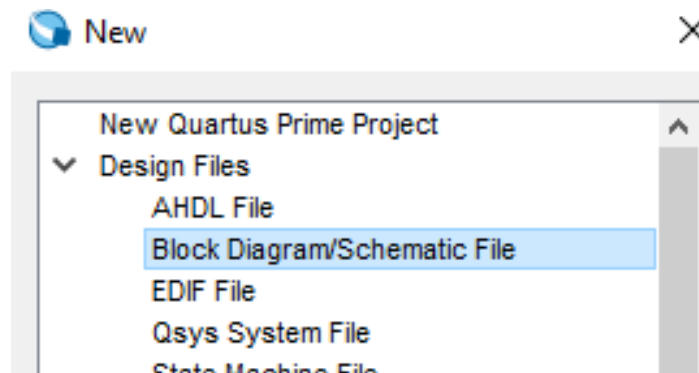


**Imagen 14: Circuito RTL**

El **RTL Viewer** en Quartus te muestra el **esquema lógico sintetizado** (el circuito que el compilador generó a partir de tu código VHDL). Pero **el RTL Viewer no es un simulador**, solo es una herramienta de verificación visual del diseño. Entonces hay que simularlo y ver si es correcto.

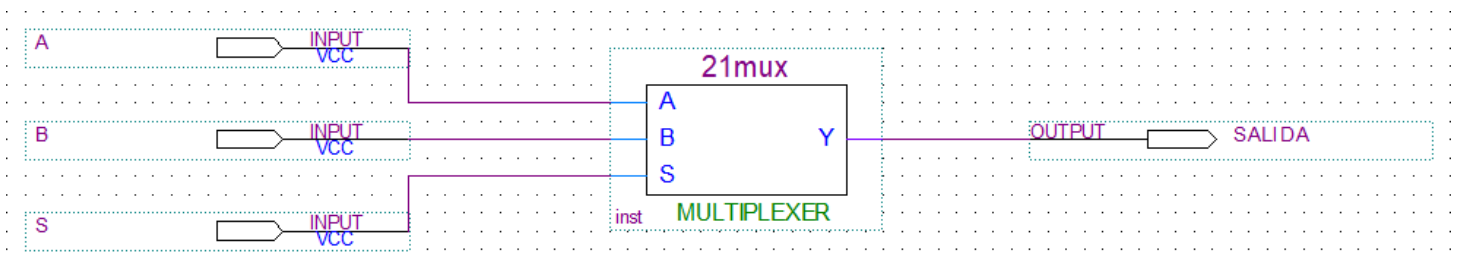
**7. Una compilación exitosa no garantiza un funcionamiento satisfactorio, pues podrían existir errores de lógica. Por lo anterior es conveniente simular comportamiento del circuito. Para ello se procederá a realizar la simulación del mismo Considere los mismos pasos que se siguieron en la practica 1**

**Paso 1:** Seleccionamos “File” luego “New” y damos clic en “Block Diagram/Schematic File”



**Imagen 15: Block Diagram/Schematic File**

**Paso 2:** Ármanos el circuito

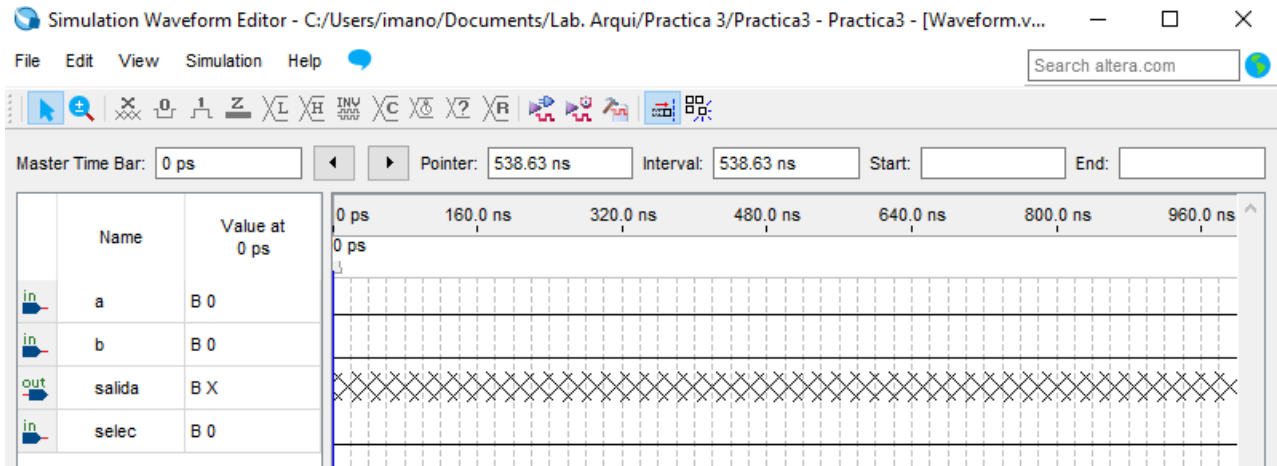


**Imagen 16: Circuito**

## Simulación 1

Como quiero hacer la simulación en Waveform Editor, entonces seguimos el siguiente paso.

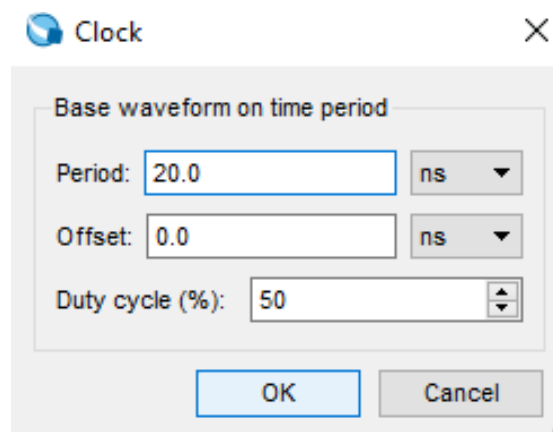
**Paso 3:** En la ventana New, en la sección Verification/Debugging Files seleccionar la opción University Program VWF.



**Imagen 17: University Program VWF**

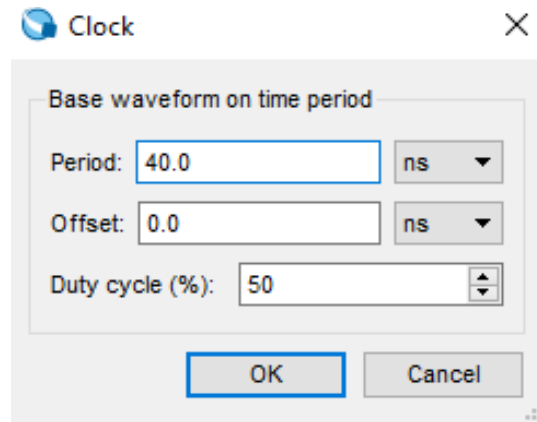
Ejemplo de configuración para probar todas las combinaciones:

4. a: configúralo como Clock con periodo 20.



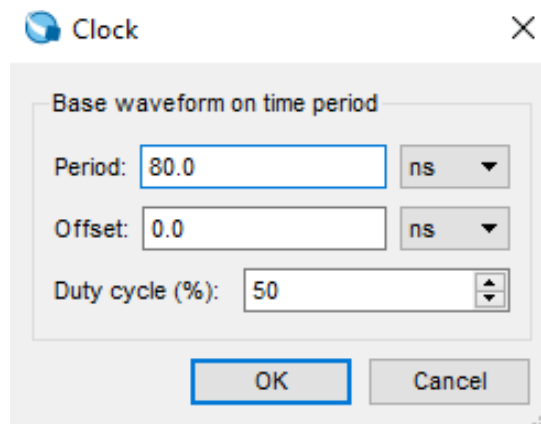
**Imagen 18: Clock de a**

5. b: configúralo como Clock con periodo 40.



**Imagen 19: Clock de b**

6. selec: configúralo como Clock con periodo 80.



**Imagen 20: Clock de selec**

Esto generará automáticamente las 8 combinaciones posibles de (a, b, selec).

| Tiempo | selec | a | b | Salida |
|--------|-------|---|---|--------|
| 0      | 0     | 0 | 0 | 0      |
| 10     | 0     | 1 | 0 | 1      |
| 20     | 0     | 0 | 1 | 0      |
| 30     | 0     | 1 | 1 | 1      |
| 40     | 1     | 0 | 0 | 0      |
| 50     | 1     | 1 | 0 | 0      |
| 60     | 1     | 0 | 1 | 1      |
| 70     | 1     | 1 | 1 | 1      |

## 1. Qué significa configurar como Clock

Cuando configuras una señal como **Clock**, Quartus/ModelSim hace que esa entrada sea una onda periódica que alterna entre 0 y 1 según el **periodo** que le pongas.

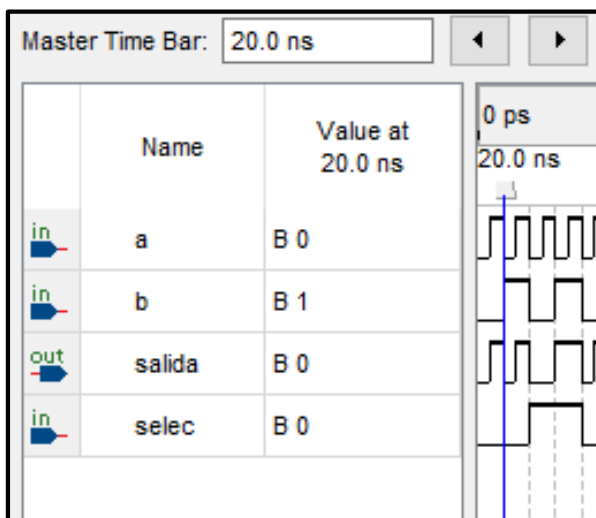
- Periodo 20 = cambia más rápido (alta frecuencia).
- Periodo 80 = cambia más lento (baja frecuencia).

## 2. Cómo funciona en tu ejemplo

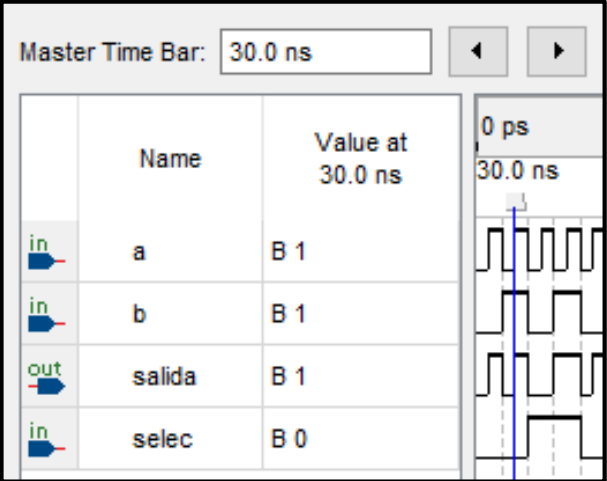
- a0 con periodo **20** → cambia de 0 a 1 cada 10 unidades de tiempo.
- b0 con periodo **40** → cambia cada 20 unidades de tiempo (la mitad de rápido que a0).
- select con periodo **80** → cambia cada 40 unidades de tiempo (todavía más lento).

Esto genera una especie de **contador binario automático**

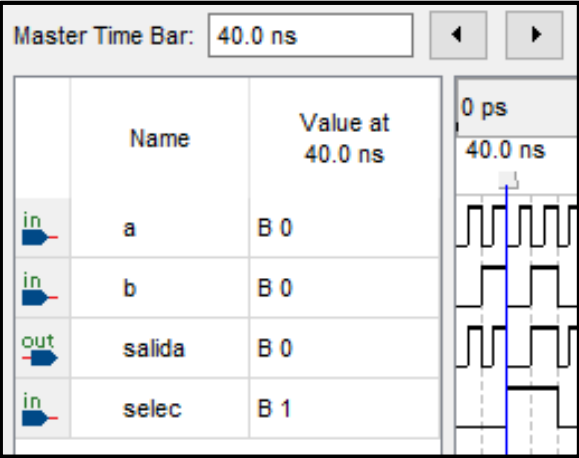
Veamos los resultados en la simulación 1:



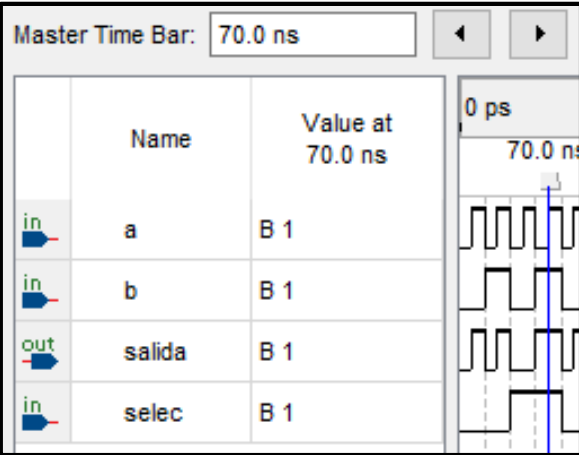
| Tiempo | selec | a | b | Salida |
|--------|-------|---|---|--------|
| 20     | 0     | 0 | 1 | 0      |



| Tiempo | selec | a | b | Salida |
|--------|-------|---|---|--------|
| 30     | 0     | 1 | 1 | 1      |



| Tiempo | selec | a | b | Salida |
|--------|-------|---|---|--------|
| 40     | 1     | 0 | 0 | 0      |



| Tiempo | selec | a | b | Salida |
|--------|-------|---|---|--------|
| 70     | 1     | 1 | 1 | 1      |



## Simulación 2

Utilizamos la otra simulación que es la misma interpretación

| Name   | Value    | Kind   | Mode     |
|--------|----------|--------|----------|
| a      | Not L... | Signal | In       |
| b      | Not L... | Signal | In       |
| selec  | Not L... | Signal | In       |
| salida | Not L... | Signal | Out      |
| gnd    | Not L... | Signal | Internal |

|                   |   |
|-------------------|---|
| /practica3/a      | U |
| /practica3/b      | U |
| /practica3/selec  | U |
| /practica3/salida | 0 |

En esta imagen vemos las entradas y salidas de la simulación. Ahora toca asignarles los mismo valores de entrada para a0, b0, c0, como se muestra a continuación:

Define Clock

Clock Name

sim:/practica3/a

offset

0

Duty

50

Period

20

Cancel

Logic Values

High: 1 Low: 0

First Edge

☒ Rising ☐ Falling

OK

Cancel

Define Clock

Clock Name

sim:/practica3/b

offset

0

Duty

50

Period

40

Cancel

Logic Values

High: 1 Low: 0

First Edge

☒ Rising ☐ Falling

OK

Cancel

Define Clock

Clock Name

sim:/practica3/selec

offset

0

Duty

50

Period

80

Cancel

Logic Values

High: 1 Low: 0

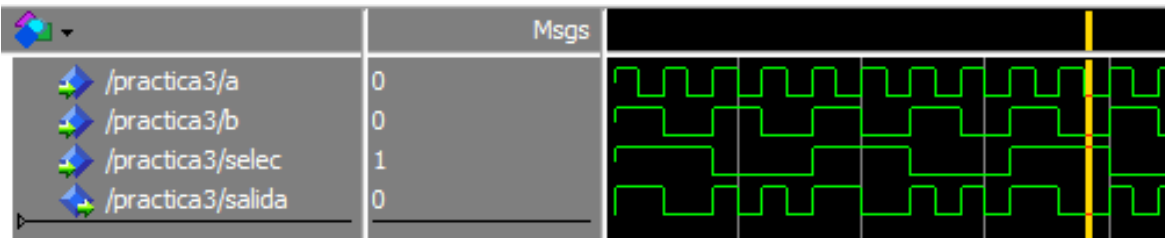
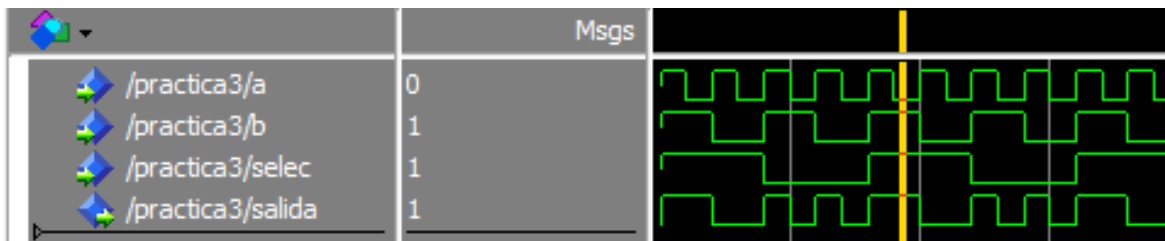
First Edge

☒ Rising ☐ Falling

OK

Cancel

La tabla de resultados del **multiplexor 2:1 (MUX)** se interpreta verificando que la salida responde exactamente a la lógica definida en el código.



### Análisis por casos:

1. **Cuando selec = 0** → la salida debe copiar el valor de a.

o Ejemplo: en el tiempo 10, selec = 0, a = 1, b = 0, por lo tanto la salida = 1 (igual que a).

2. **Cuando selec = 1** → la salida debe copiar el valor de b.

o Ejemplo: en el tiempo 60, selec = 1, a = 0, b = 1, por lo tanto la salida = 1 (igual que b).

### Qué confirma la tabla:

- La salida **no depende de ambas entradas al mismo tiempo**, sino únicamente de la que el selector (selec) indique.
- Con el patrón de Clocks de distinto periodo, se recorren automáticamente las 8 combinaciones posibles de entradas (a, b, selec), y en todos los casos la salida coincide con la **tabla de verdad del MUX**.

En conclusión, la tabla muestra que el diseño en VHDL funciona correctamente: el multiplexor selecciona entre a y b dependiendo del valor de selec.

## Conclusión

La práctica permitió validar con éxito la implementación de un multiplexor 2:1 en VHDL, cuya funcionalidad fue corroborada mediante el análisis de formas de onda en **Quartus Prime**. La estrategia de configurar las entradas como señales periódicas facilitó la inspección exhaustiva de todas las combinaciones lógicas, confirmando que la salida responde fielmente a la señal de selección según la tabla de verdad teórica. Este proceso subrayó una lección fundamental en el diseño digital: la simulación es el único método fiable para garantizar que la arquitectura descrita cumple con el comportamiento esperado. A pesar de la complejidad inicial en la sincronización de periodos en el simulador, la correcta interpretación de los diagramas de tiempos permitió consolidar el dominio de las herramientas de verificación.

## Referencias

1. Intel. (2025). *Intel® Quartus® Prime Design Software*. Intel. Recuperado el 10 de septiembre de 2025 de <https://www.intel.com/content/www/us/en/software/programmable/quartus-prime/overview.html>
2. FPGA4student. (2025). *VHDL tutorial: Full Adder*. FPGA4student. Recuperado el 10 de septiembre de 2025 de <https://www.fpga4student.com/2017/01/vhdl-code-for-full-adder.html>
3. Allaboutcircuits. (2025). *Introduction to VHDL for Digital Design*. All About Circuits. Recuperado el 10 de septiembre de 2025 de <https://www.allaboutcircuits.com/technical-articles/introduction-to-vhdl-for-digital-design>